

for data Modelling are listed below.

Scalability: Scalability refers to the increase in application workload due to increase in traffic. Applications should be designed to handle and perform well when the usage of the application increases. From the NoSQL perspective, it means that collections and data entities should be modelled based on the current and future demand for the application. There should not be non-availability or degradation of performance due to an increase in the number of users or transactions in the database.

Performance: Database Modelling is about trade-offs between security, availability, scalability and performance. A right balance between these architectural blocks helps in creating the optimal database design for an application. A data model helps in understanding the read and write needs of an application and also helps in deciphering data updates patterns frequently while some remains static most of the time. It helps in creating an application that performs as expected with an increased workload over time.

Application needs: Different applications have different demands from the database. Some are read-intensive while others are write-centric. Applications can be OLTP based or meant for reporting. Some applications have multiple facets built into them. Different data Modelling strategies should be used based on the nature of the application. Reporting applications are read heavy and writes should not reduce their performance, while transactional systems should be able to read and write with equal ease.

Data consistency: Data consistency helps in reducing redundant data, and understanding relationships, their update patterns and taxonomy. It helps in storing only the required data in its correct form.

Capacity: Each document has a defined size and, generally, performance falls with re-allocation of size to it. Data Modelling helps in creating optimal sized documents, reducing redundant data in each document. It also helps in identifying the overall capacity needs of the database.

MongoDB support for data Modelling

MongoDB provides advanced constructs for enabling data Modelling. It provides References and Embedment for defining structure and relationships of documents. It is important to understand these two constructs before getting into data Modelling.

References

Relationships are defined based on matching data contained in columns in different collections. In MongoDB, these relationships are defined based on semantics. The MongoDB engine does not enforce this relationship, and it is completely dependent on the application to implement and respect this relationship while reading and writing data in collections. References store the relationships between data by including links or references from one document to another.

Applications can resolve these references to access the related data. Data models based on References are also known as Normalised data models. The example illustrated next shows the Users document references in a *placeOfBirth* document.

```
{
  "_id": "Ritesh",
  "name": "Ritesh Modi",
  "placeOfBirth": "RiteshPOB"
}

{
  "PlaceOfBirth": {
    "_id": "RiteshPOB"
    "street": "123 xyz street",
    "city": "xyzcity",
    "state": "xyzState",
    "zip": "12345"
  }
}
```

References relationship should be used:

- To implement one-to-many relationships between documents.
- To implement many-to-many relationships between documents.
- If the referenced entities are updated frequently.
- If the referenced entities grow indefinitely.

Embedment

Embedded relationships in documents refer to storing related documents within a original document. The related data is part of the schema of embedding documents. In effect, the entire data is stored together within a single document, with related data stored as an array or sub-object. Data models based on Embedment are also known as De-Normalised data models. The example illustrated next shows the *placeOfBirth* entity embedded in the Users document.

```
{
  "_id": "Ritesh",
  "name": "Ritesh Modi",
  "placeOfBirth": {
    "street": "123 xyz Street",
    "city": "xyzcity",
    "state": "xyzState",
    "zip": "12345"
  }
}
```

Embedded documents should be used when:

- There is a contained relationship between entities.
- The embedded entity is an integral part of the document.
- The embedded entities are not updated frequently.